

Unit 9 – B-Tree

CS 201 - Data Structures II

Spring 2024

Habib University

Syeda Saleha Raza

About Memory Management

- Memory Word (4, 8, 16 bytes)
- Memory addresses
- Blocks
- Fragmentation
- Garbage Collection

Memory Hierarchy

- Registers
- Cache
- Internal Memory (Main memory)
- External Memory (Disks, drives, DVD, Tape)

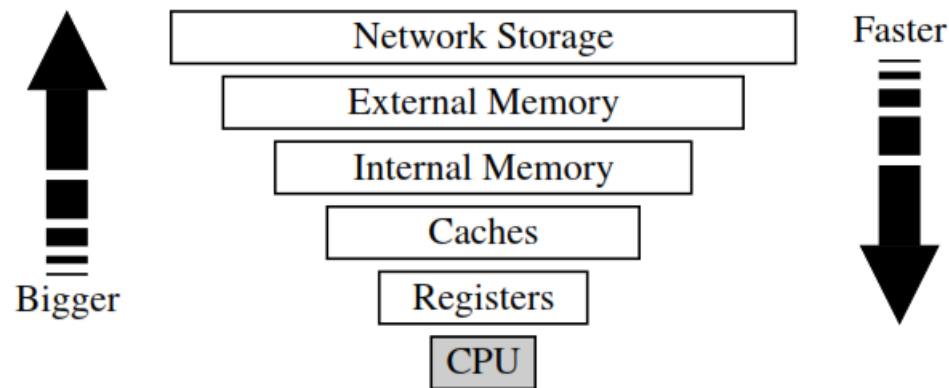
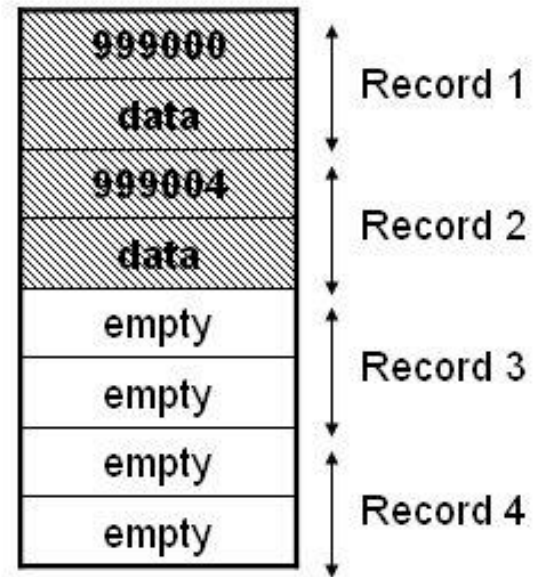


Figure 15.1: The memory hierarchy.

Introduction

- Data is stored on disk (i.e., secondary memory) in blocks.
- A block is the smallest amount of data that can be accessed on a disk.
- Each block has a fixed number of bytes – typically 512, 1024, 2048, 4096 or 8192 bytes
- Each block may hold many data records.



Motivation for Studying B-Trees

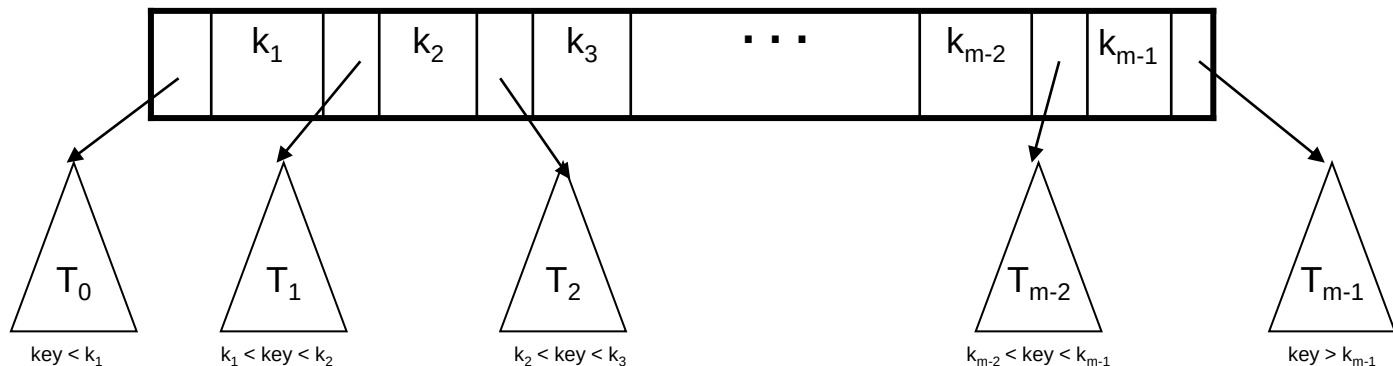
- A disk access is very expensive compared to a typical computer instruction (mechanical limitations) - One disk access is worth about 200,000 instructions.
- When data is too large to fit in main memory the number of disk accesses becomes important.
- Many algorithms and data structures efficient for manipulating data in primary memory are not efficient for manipulating large data in secondary memory.
- AVL trees are not suitable for representing huge tables residing in secondary memory.
- The height of an AVL tree increases, and hence the number of disk accesses required to access a particular record increases, as the number of records increases.

Locality of reference properties

- **Temporal locality:** If a program accesses a certain memory location, then there is increased likelihood that it accesses that same location again in the near future. For example, it is common to use the value of a counter variable in several different expressions, including one to increment the counter's value. In fact, a common adage among computer architects is that a program spends 90 percent of its time in 10 percent of its code.
- **Spatial locality:** If a program accesses a certain memory location, then there is increased likelihood that it soon accesses other locations that are near this one. For example, a program using an array may be likely to access the locations of this array in a sequential or near-sequential manner.

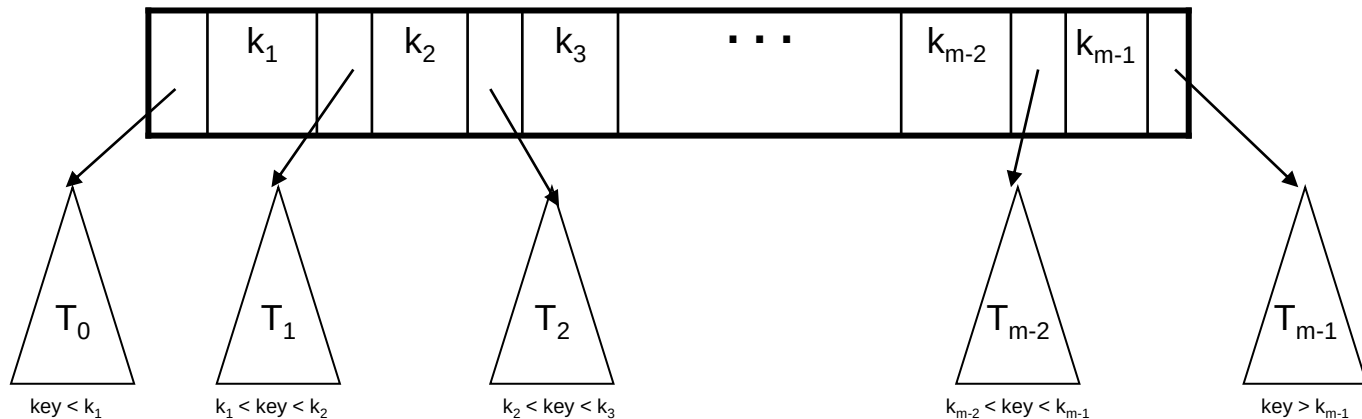
Multiway Trees

- A multi-way (or m -way) search tree of order m is a tree in which
 - Each node has at-most m subtrees, where the subtrees may be empty.
 - Each node consists of at least 1 and at most $m - 1$ distinct keys
 - The keys in each node are sorted.



Multiway Trees

- The keys and subtrees of a non-leaf node are ordered as:
 $T_0, k_1, T_1, k_2, T_2, \dots, k_{m-1}, T_{m-1}$ such that:
 - All keys in subtree T_0 are less than k_1 .
 - All keys in subtree $T_i, 1 \leq i \leq m-2$, are greater than k_i but less than k_{i+1} .
 - All keys in subtree T_{m-1} are greater than k_{m-1} .



B-Tree Height

- Let $d = \lceil m / 2 \rceil$. Therefore, the maximum height of a B-tree is:
$$h_{max} = \lfloor \log_d ((n + 1) / 2) \rfloor$$
 - If $m = 300$ and $n = 16,000,000$ then $h \approx 4$.
 - Thus, in the worst case finding a key in such a B-tree requires 3 disk accesses (assuming the root node is always in main memory).
- The average number of comparisons for an AVL tree with n keys is $\log_2 n$, where n is large.
 - If $n = 16,000,000$ the average number of comparisons is 24.
 - Thus, in the average case, finding a key in such an AVL tree requires 24 disk accesses.

B-Tree

- There are certain conditions that must be true for a B-tree:
 - The height of the tree must remain as minimum as possible.
 - Above the leaves of the tree, there should not be any empty subtrees.
 - The leaves of the tree must occur at the same level.
 - All nodes must have some minimum number of children except leaf nodes.

Example



Why use B-Tree?

- Reduces the number of reads made on the disk
- B Trees can be easily optimized to adjust its size (that is the number of child nodes) according to the disk size
- It is a specially designed technique for handling a bulky amount of data.
- It is a useful algorithm for databases and file systems.
- A good choice to opt when it comes to reading and writing large blocks of data

B-Tree

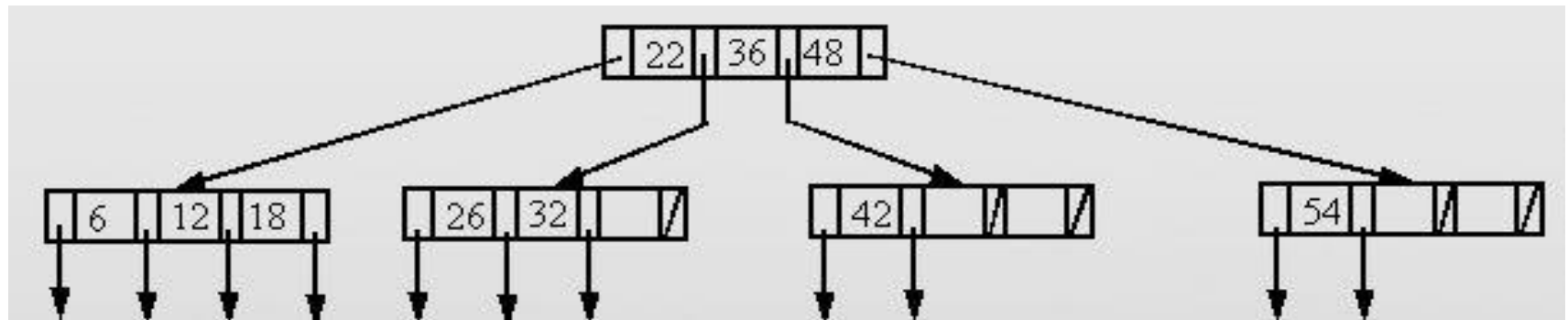
- A B-tree of order m (or branching factor m), where $m > 2$, is either an empty tree or a multiway search tree with the following properties:
 - The root is either a leaf or it has at least two non-empty subtrees and at most m non-empty subtrees.
 - Each non-leaf node, other than the root, has at least $\lceil m / 2 \rceil$ non-empty subtrees and at most m non-empty subtrees. ($\lceil x \rceil$ is the lowest integer $\geq x$).
 - The number of keys in each non-leaf node is one less than the number of non-empty subtrees for that node.
 - All leaf nodes are at the same level; that is the tree is perfectly balanced.

B-Tree: Minimum and Maximum Nodes

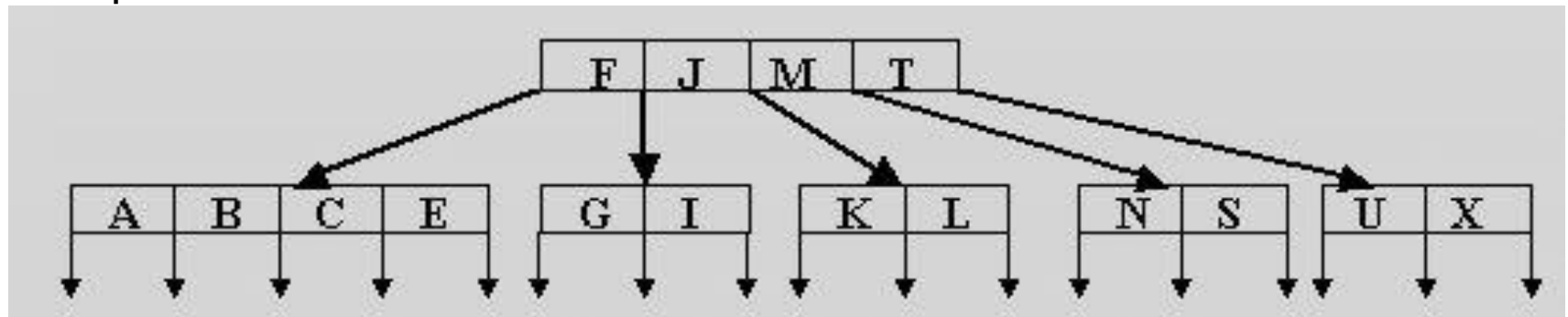
	Root Node	Non-Root Node
Minimum Number of Keys	1	$\lceil m / 2 \rceil - 1$
Minimum Number of Non-Empty Subtrees	2	$\lceil m / 2 \rceil$ (Not applicable on leaf node)
Maximum Number of Keys	$m - 1$	$m - 1$
Maximum Number of Non-Empty Subtrees	m	m

B-Tree Examples

Example: A B-tree of order 4



Example: A B-tree of order 5



2-3-4 OR 2-4 Tree

- 2-4 tree (or 2-3-4 tree) is a B-Tree of order 4.
- This type of tree can contain nodes of three types.

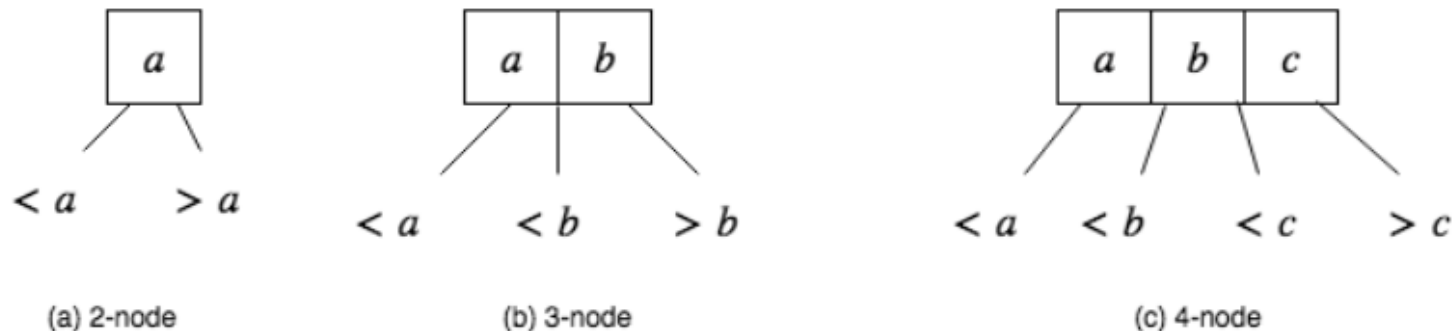


Figure 1: Illustrating node types

[2-3-4 Trees | Algorithm Tutor](#)

2-4 Tree

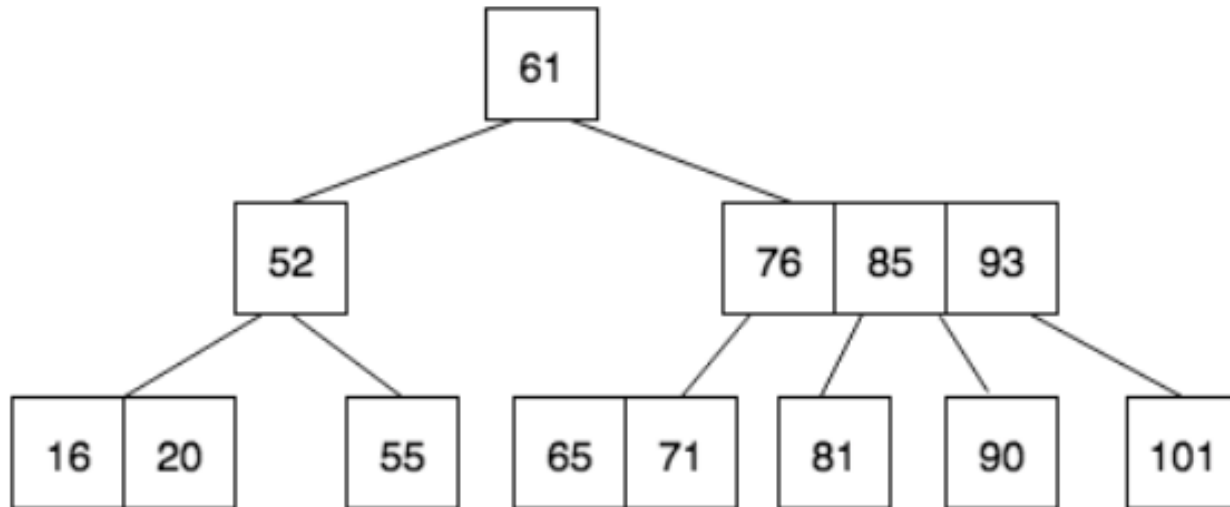
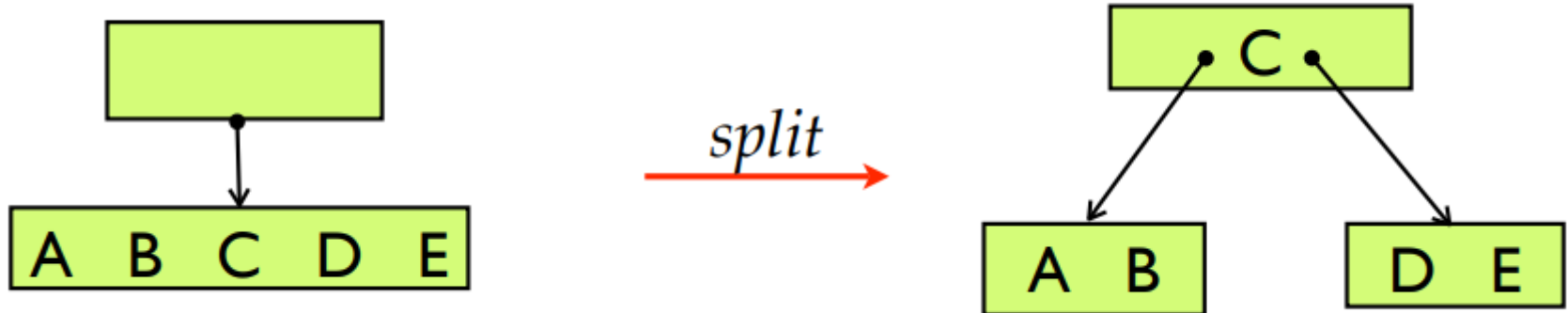


Figure 2: An example of a 2-3-4 tree

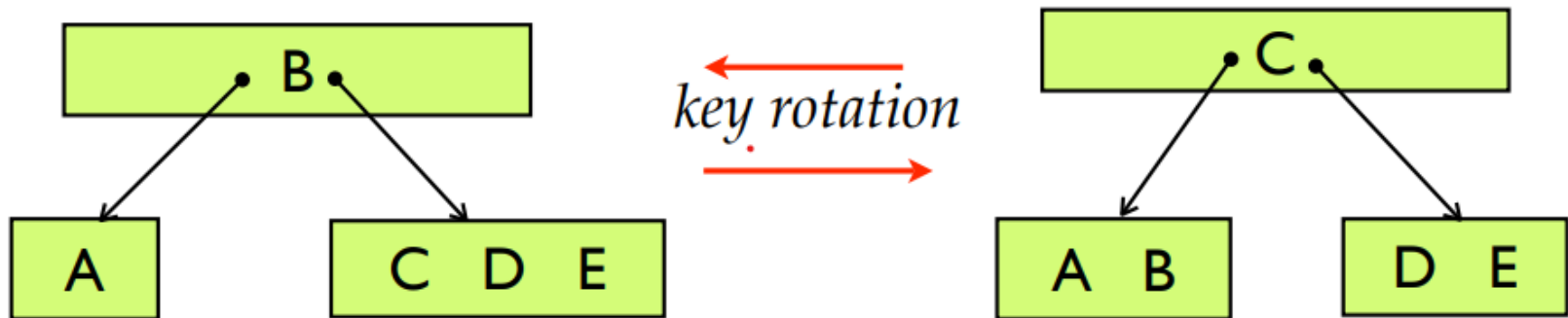
[2-3-4 Trees | Algorithm Tutor](#)

B-Tree – Insert/Delete Operations



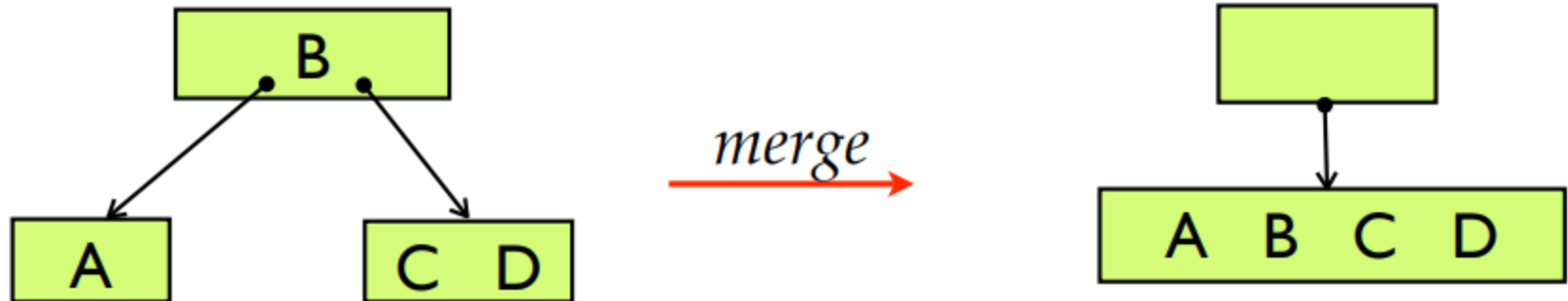
<https://www.cs.cmu.edu/~ckingsf/bioinfo-lectures/btrees.pdf>

B-Tree – Insert/Delete Operations



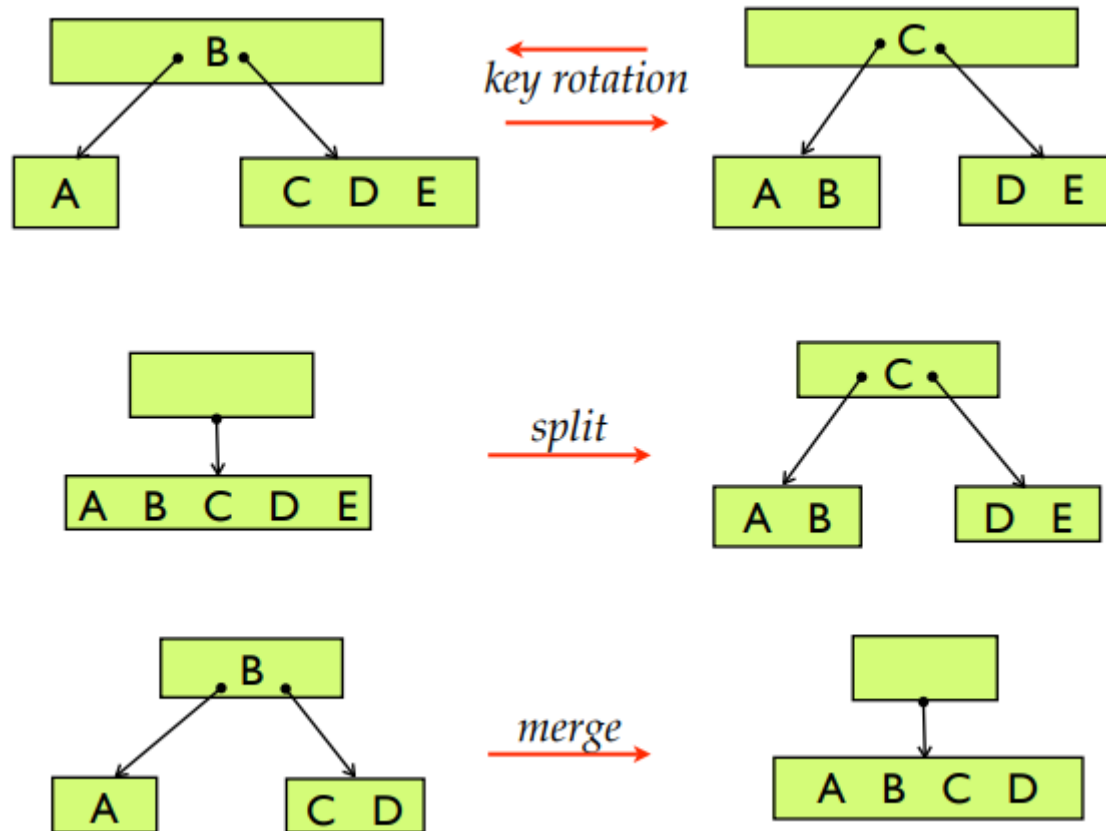
<https://www.cs.cmu.edu/~ckingsf/bioinfo-lectures/btrees.pdf>

B-Tree – Insert/Delete Operations



<https://www.cs.cmu.edu/~ckingsf/bioinfo-lectures/btrees.pdf>

Operators



<https://www.cs.cmu.edu/~ckingsf/bioinfo-lectures/btrees.pdf>

Insertion in B-Trees

- A new node is inserted in to a leaf node.
- If this insertions results in the overflow of that leaf nodes, then split the node into two, propagate the "middle" key to the parent of the node.
- If the parent overflows the process propagates upward. If the node has no parent, create a new root node.

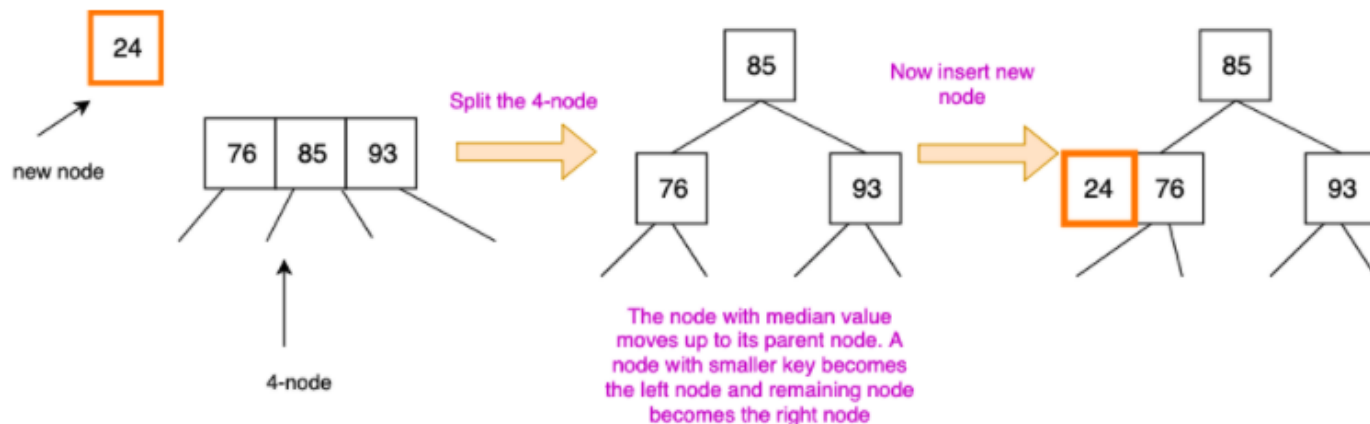
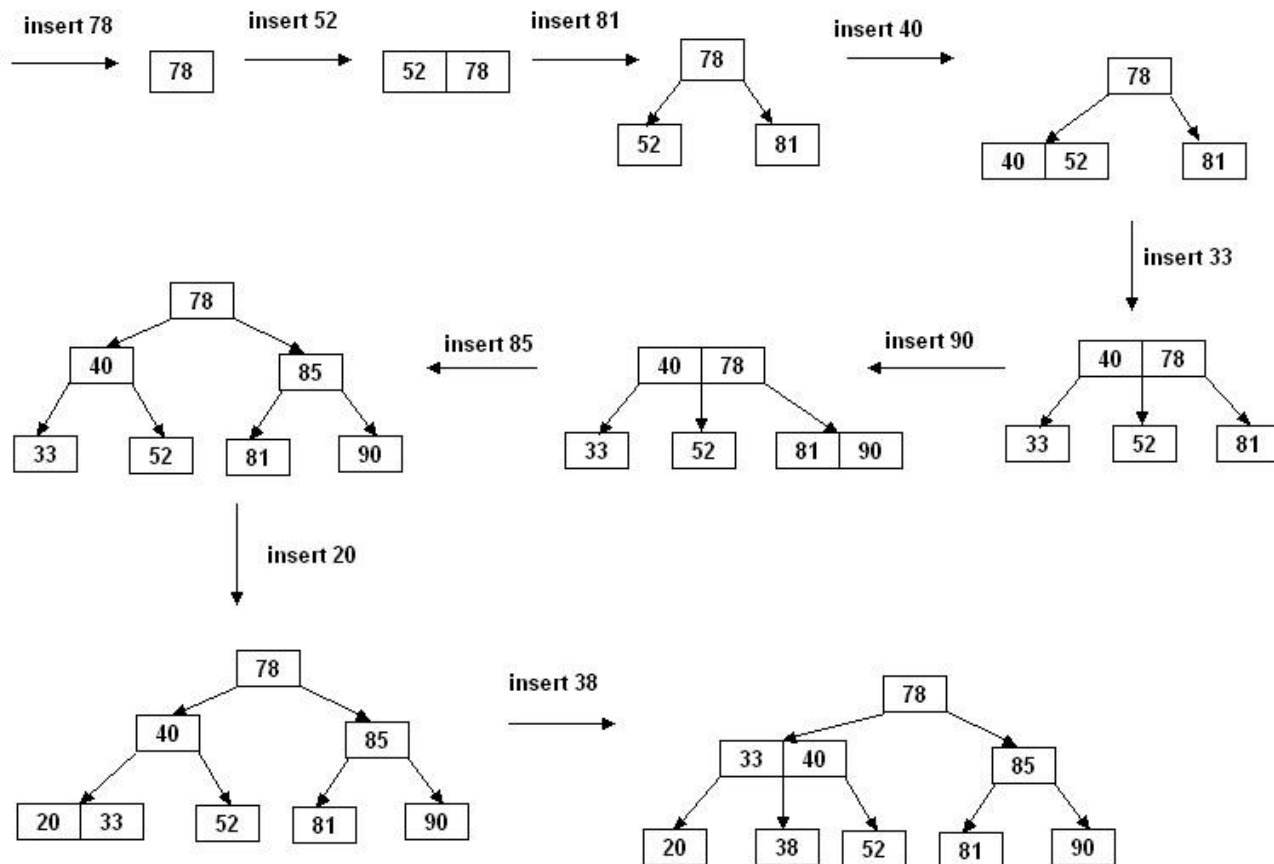


Figure 3: Splitting of a 4-node before inserting a new node.

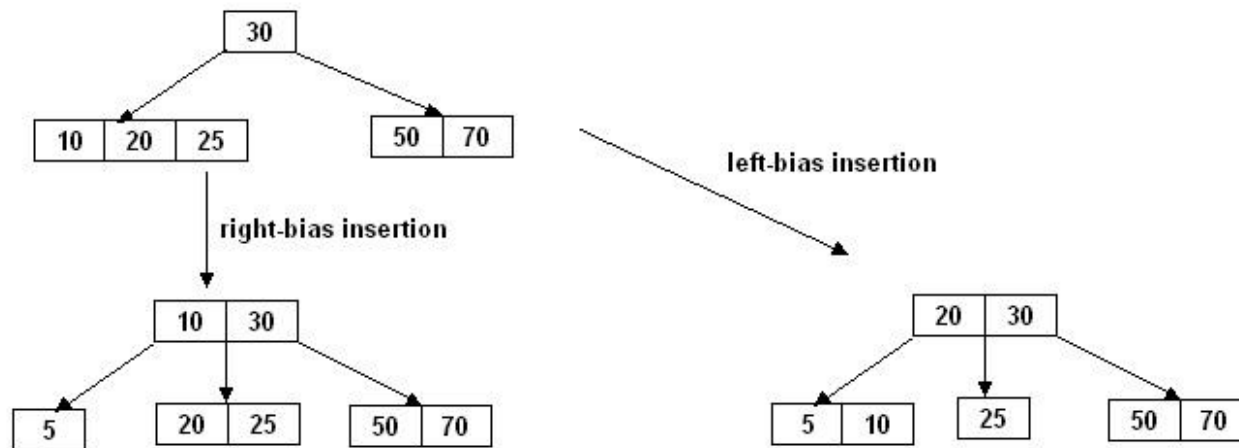
Insertion: Odd Order

- Example: Insert the keys 78, 52, 81, 40, 33, 90, 85, 20, and 38 in this order in an initially empty B-tree of order 3



Insertion: even order

- **Insertion in a B-tree of even order:** At each node the insertion can be done in two different ways:
- **right-bias:** The node is split such that its right subtree has more keys than the left subtree.
- **left-bias:** The node is split such that its left subtree has more keys than the right subtree.
- **Example:** Insert the key 5 in the following B-tree of order 4:



Example - Sequence of Inserts

Insert 76



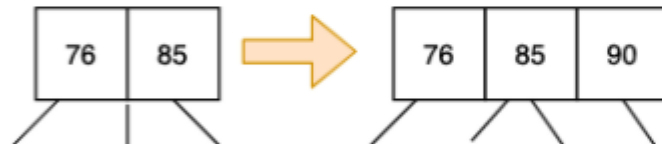
Since the tree is empty, the new node becomes the root of the tree

Insert 85



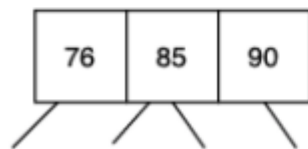
2-node becomes 3-node

Insert 90

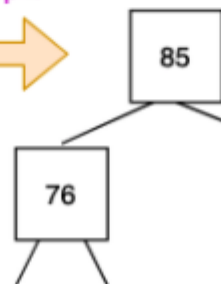


3-node becomes 4-node

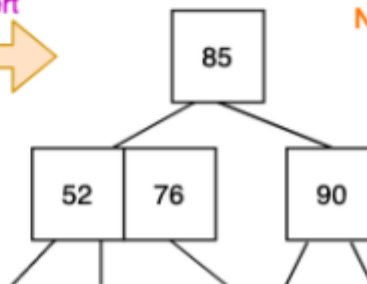
Insert 52



Split



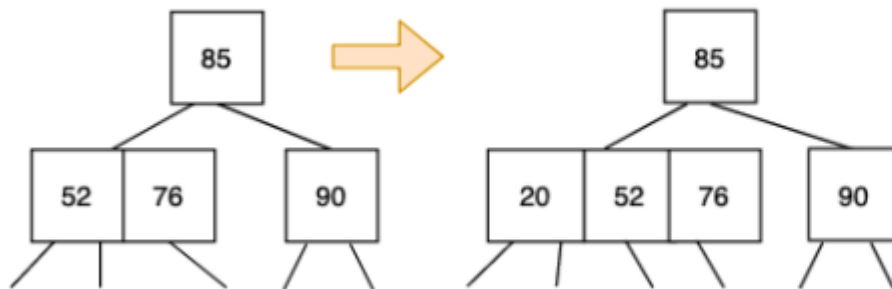
Move to the suitable leaf node and insert



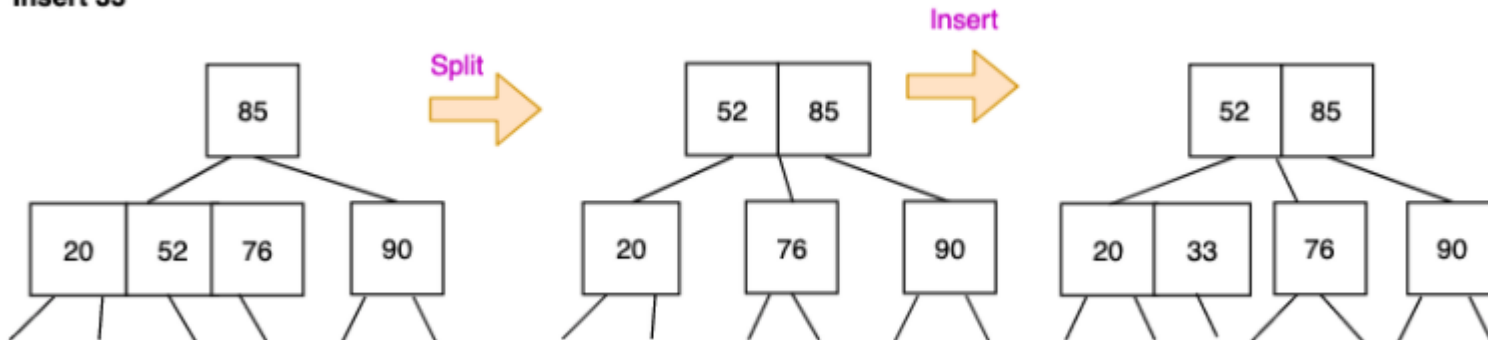
Notice the height change

Example - Sequence of Inserts

Insert 20

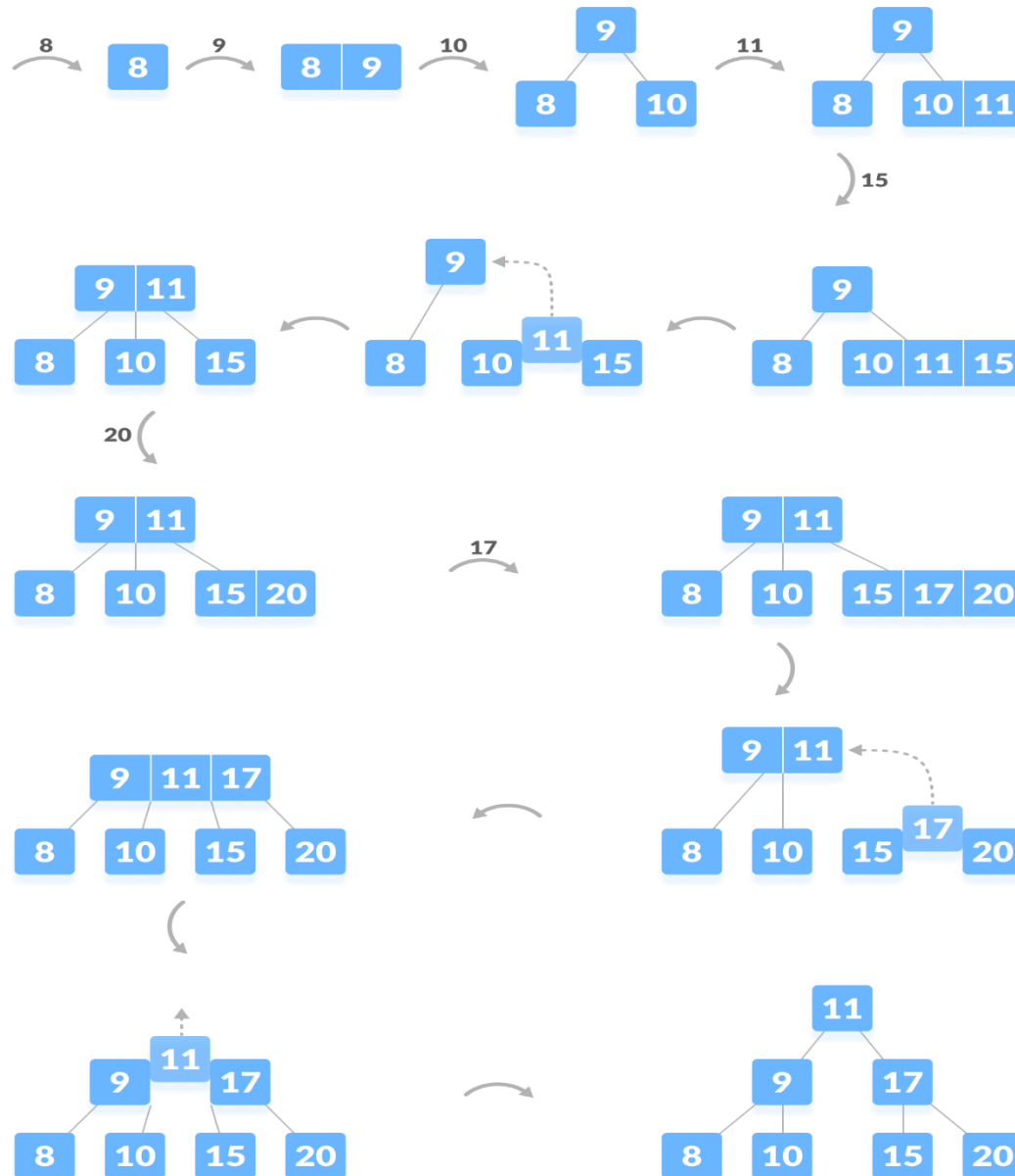


Insert 33



[2-3-4 Trees | Algorithm Tutor](#)

Insertion – Another Example



Insertion – Example to solve

- Insert the following values in a 2-4 tree.
 - 21,14,39,64,11,78,51,37,20,7,19,2,13

Insertion - Exercise

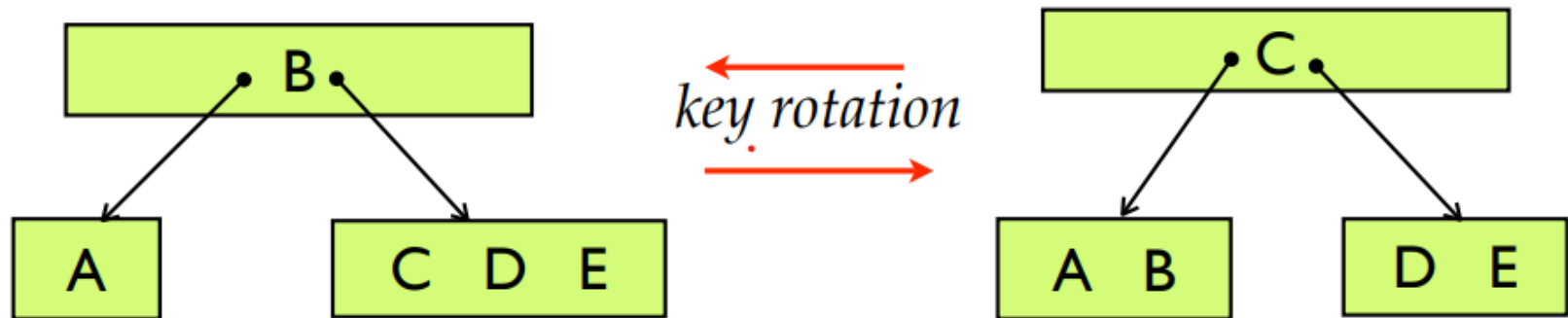
- Construct a 2-4 tree with the following values:
 - 5, 3, 21, 9, 1, 13, 2, 7, 10, 12, 4, 8

Deletion

2-3-4 Trees | Algorithm Tutor

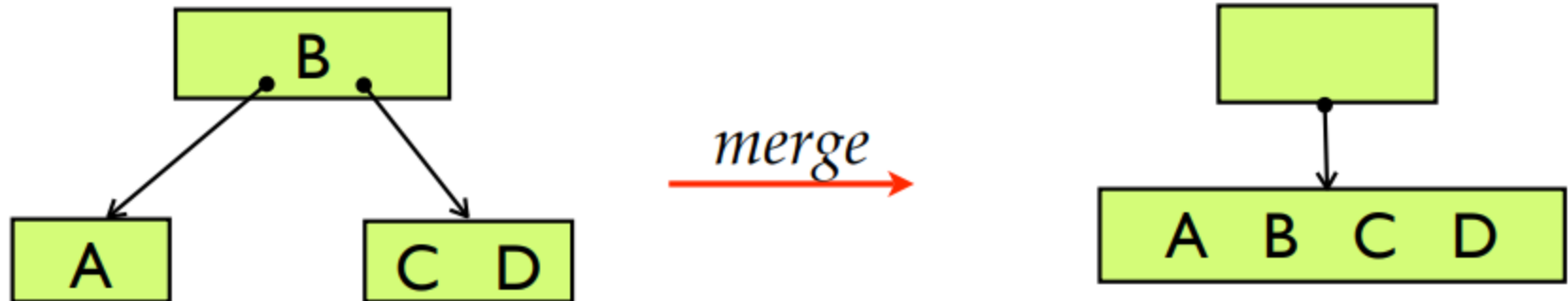
DSAP Sec. 15.3.2, 11.5

B-Tree – Insert/Delete Operations



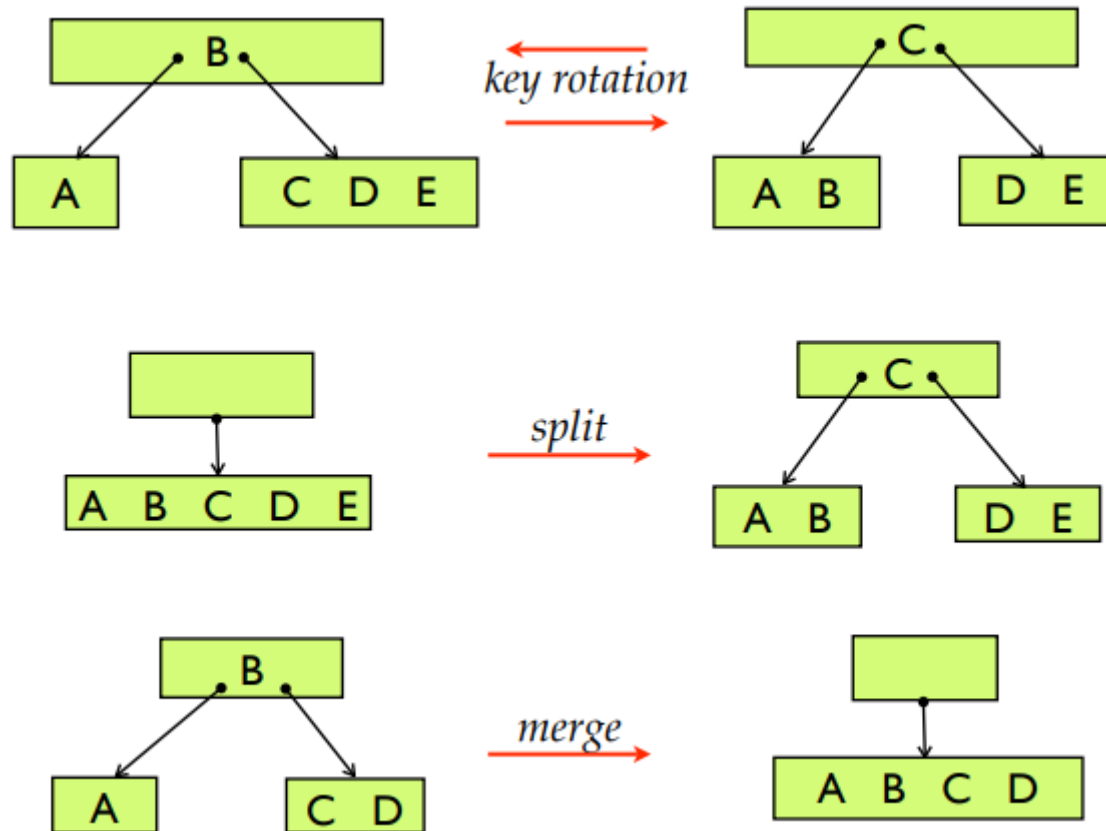
<https://www.cs.cmu.edu/~ckingsf/bioinfo-lectures/btrees.pdf>

B-Tree – Insert/Delete Operations



<https://www.cs.cmu.edu/~ckingsf/bioinfo-lectures/btrees.pdf>

Operators



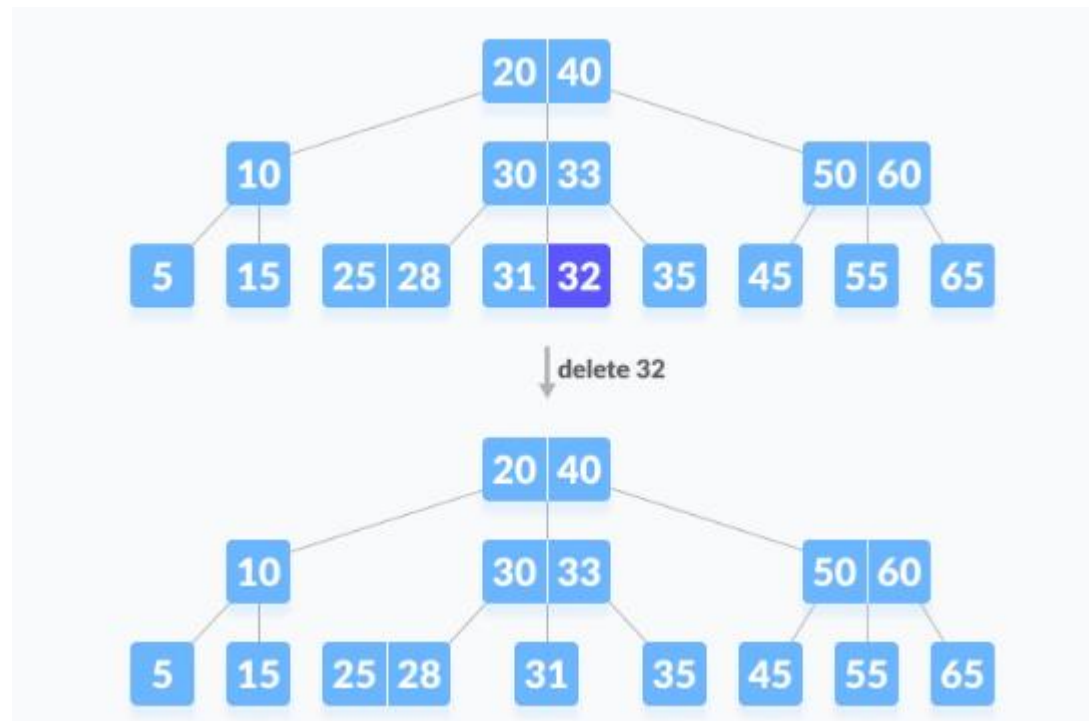
<https://www.cs.cmu.edu/~ckingsf/bioinfo-lectures/btrees.pdf>

Deleting a node from B-Tree

- leaf node
 - with more than min keys:
 - Delete the key, rest is good.
 - with min key:
 - Try to borrow a key from left or right sibling if they have more than mid child. This would happen via rotation.
 - Else
 - Merge with sibling using parent
 - If parent is also min code, apply merging at parent's level too.
- Internal node
 - if the node has more than min child
 - Delete the key, rest if good.
 - Replace by inorder predecessor/inorder successor if left/right child has more than min child
 - Else merge left and right child
 - If the node to be deleted has less than min nodes (after deletion), then merge this node with its siblings.

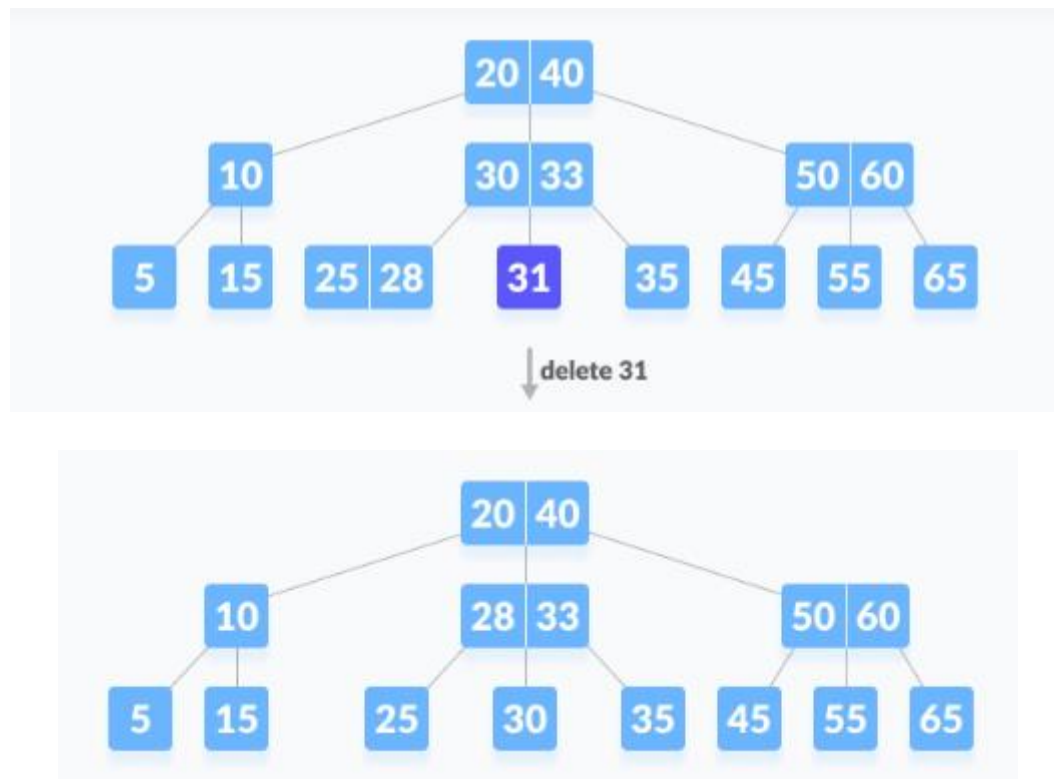
Deleting a leaf node – Case I

- The deletion of the key does not violate the property of the minimum number of keys a node should hold.



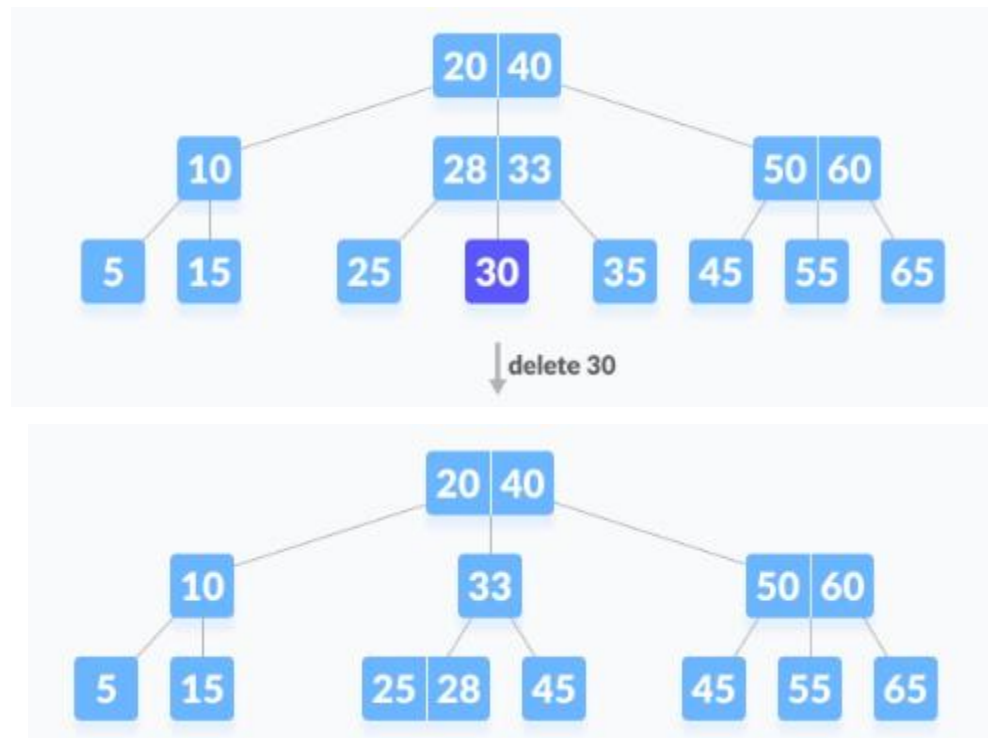
Deleting a leaf node – Case II

- The deletion of the key violates the property of the minimum number of keys a node should hold. In this case, we borrow a key from its immediate neighboring sibling node in the order of left to right.



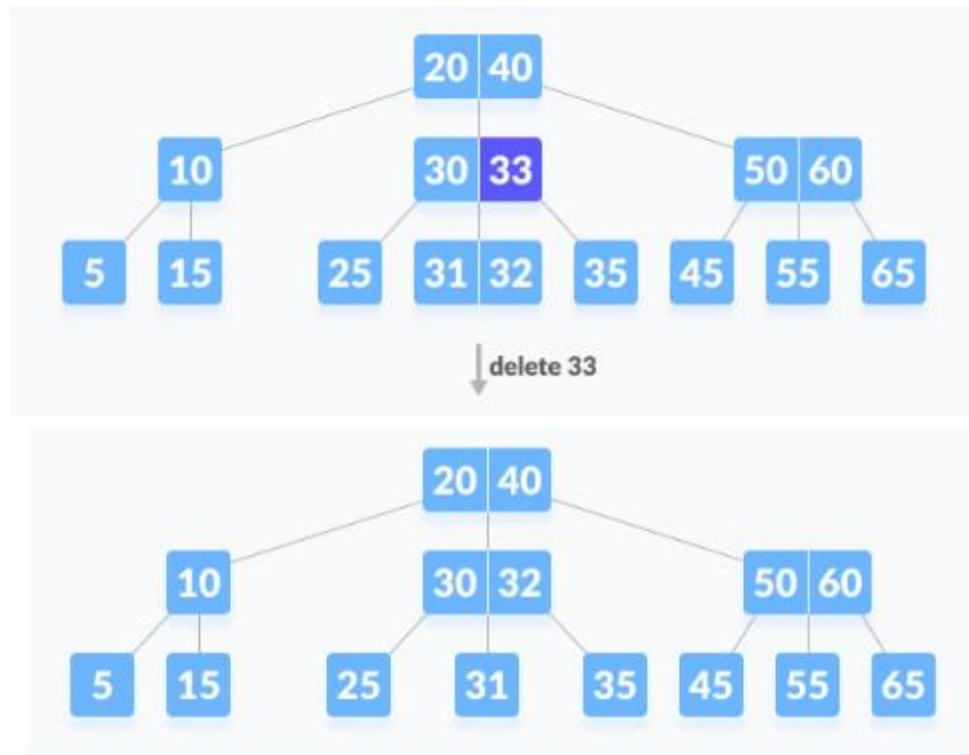
Deleting a leaf node – Case III

- If both the immediate sibling nodes already have a minimum number of keys, then merge the node with either the left sibling node or the right sibling node. **This merging is done through the parent node.**



Deleting an internal node – Case I

- The internal node, which is deleted, is replaced by an inorder predecessor if the left child has more than the minimum number of keys.

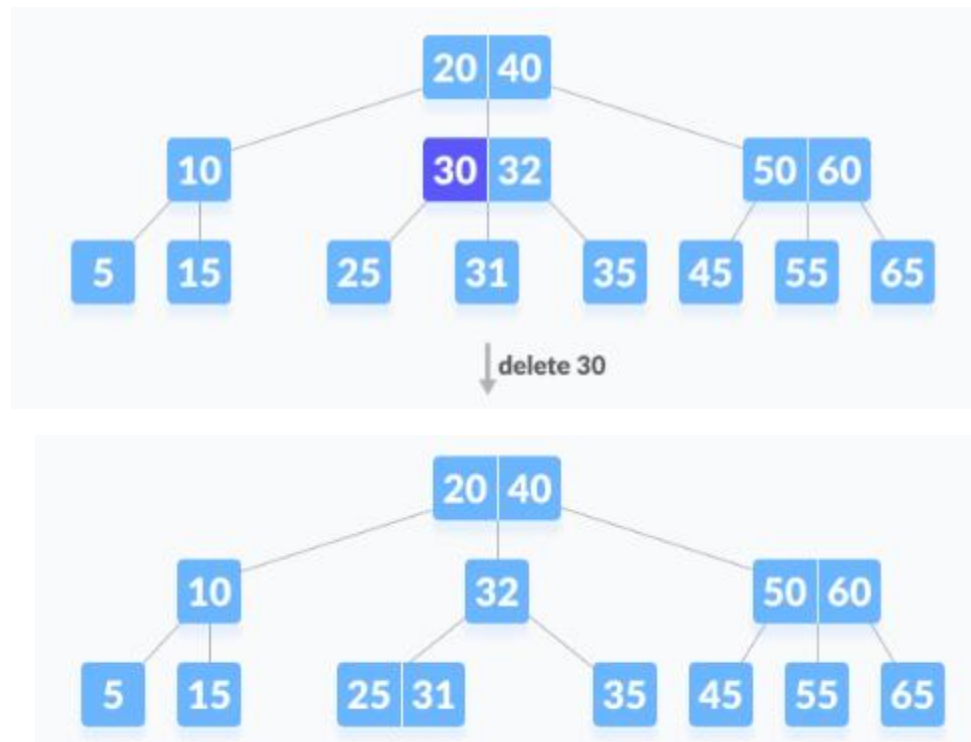


Deleting an internal node – Case II

- The internal node, which is deleted, is replaced by an inorder successor if the right child has more than the minimum number of keys.

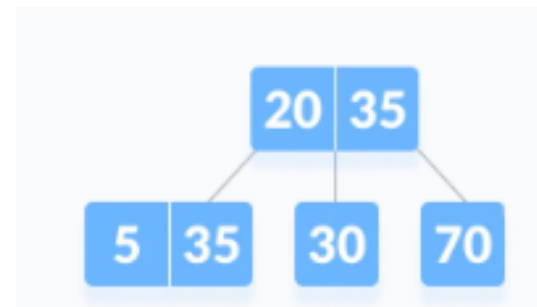
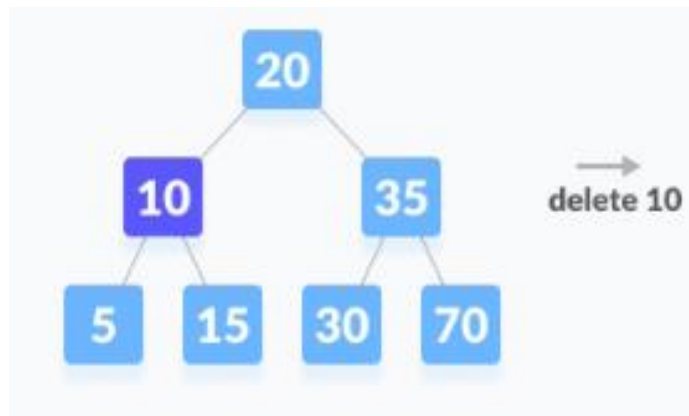
Deleting an internal code – Case III

- If either child has exactly a minimum number of keys then, merge the left and the right children.



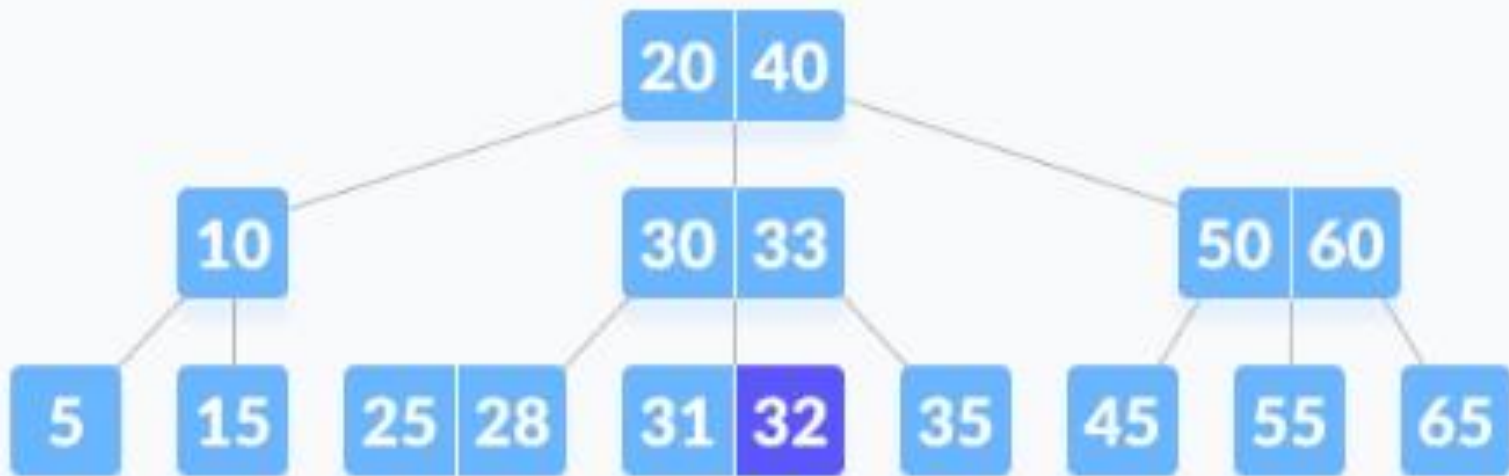
Deleting an internal node – Case IV

- In this case, the height of the tree shrinks. If the sibling also has only a minimum number of keys then, merge the node with the sibling along with the parent. Arrange the children accordingly (increasing order).



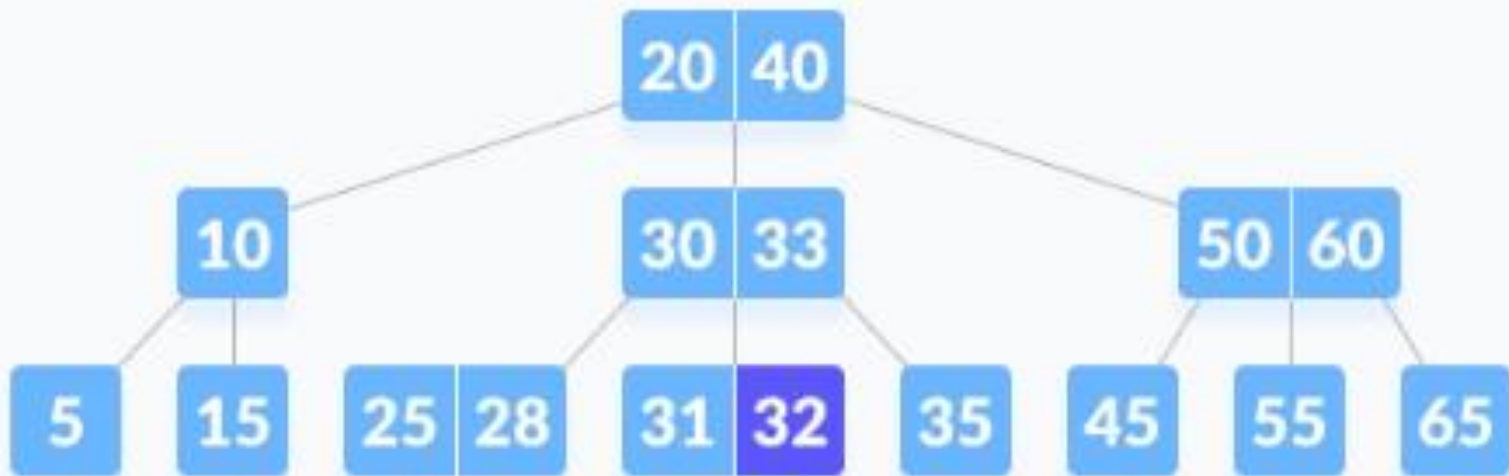
Exercise

- Delete 35.



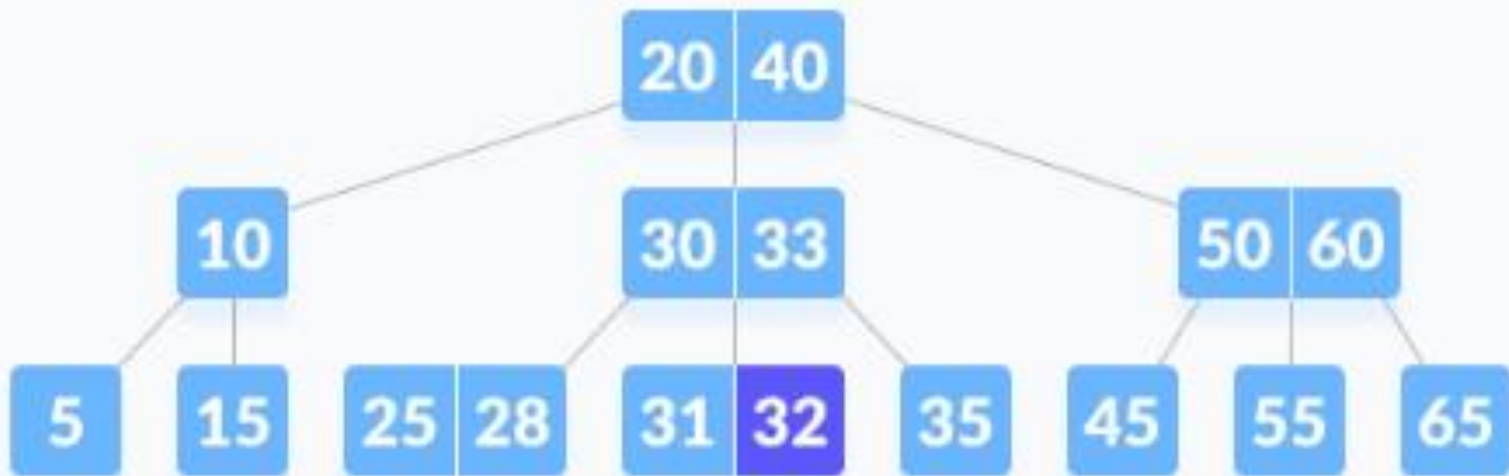
Exercise

- Delete 45.



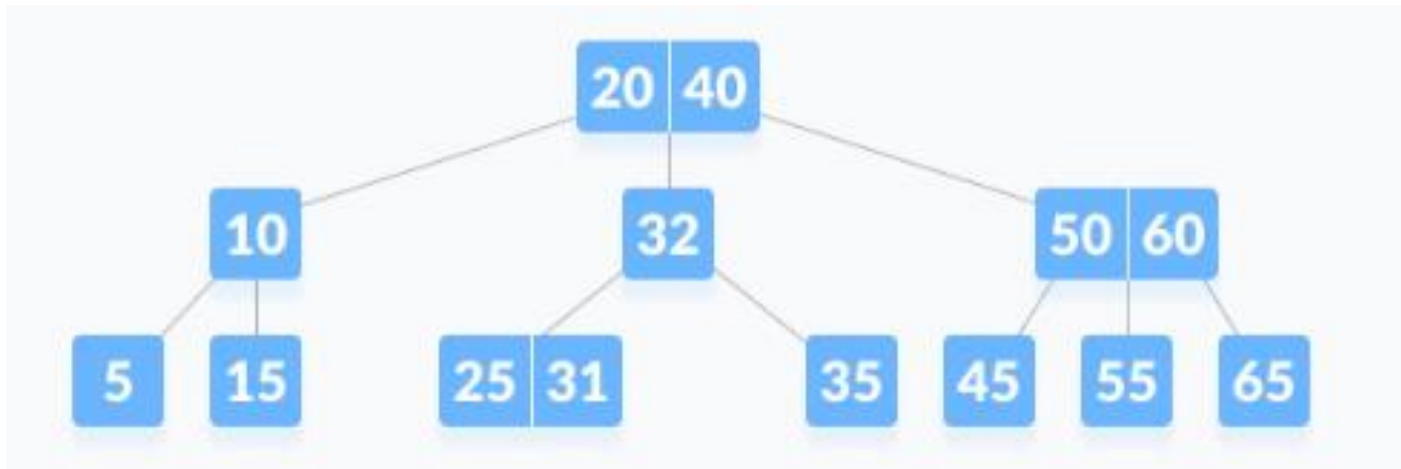
Exercise

- Delete 30.



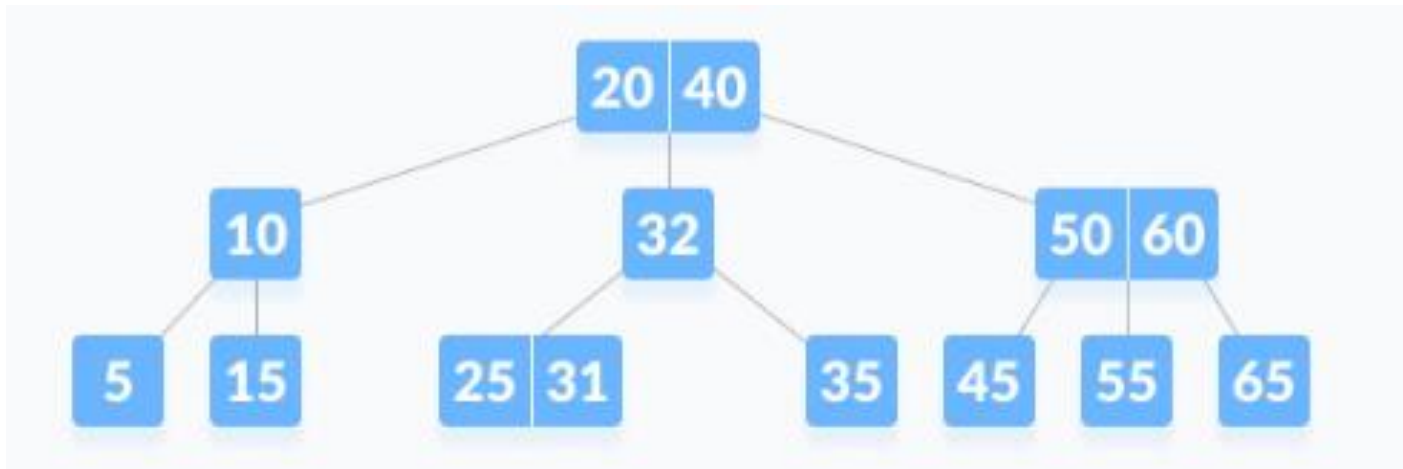
Exercise

- Delete 60.



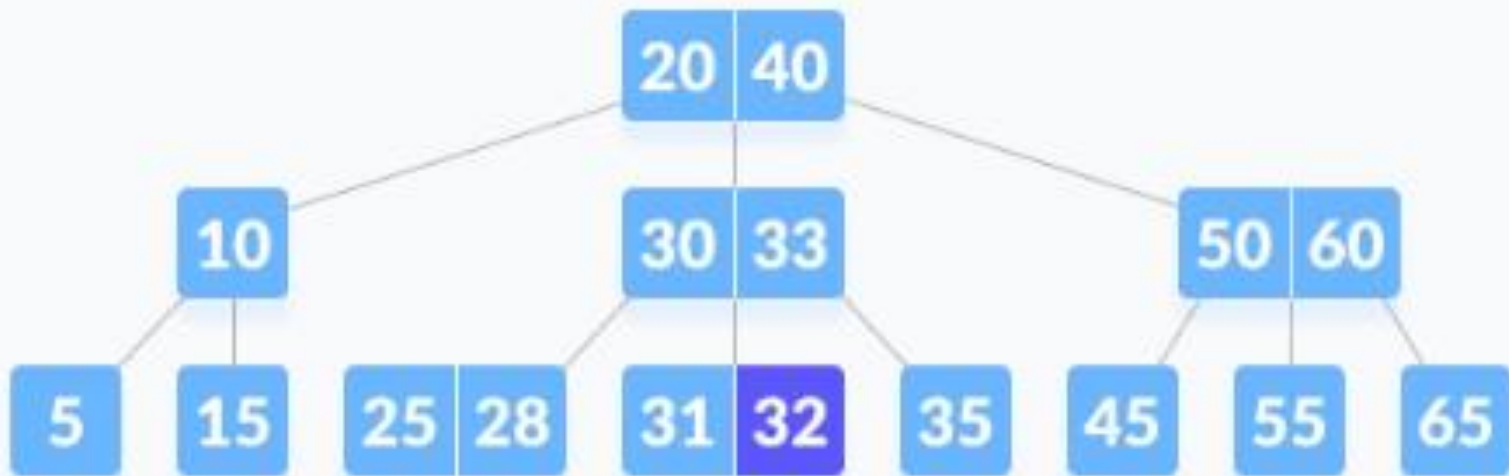
Exercise

- Delete 10.



Exercise

- Delete 5.



B-Tree

- A B-tree of order ' d ' would be an (a, b) tree with $a = \text{ceil}(d/2)$ and $b = d$.

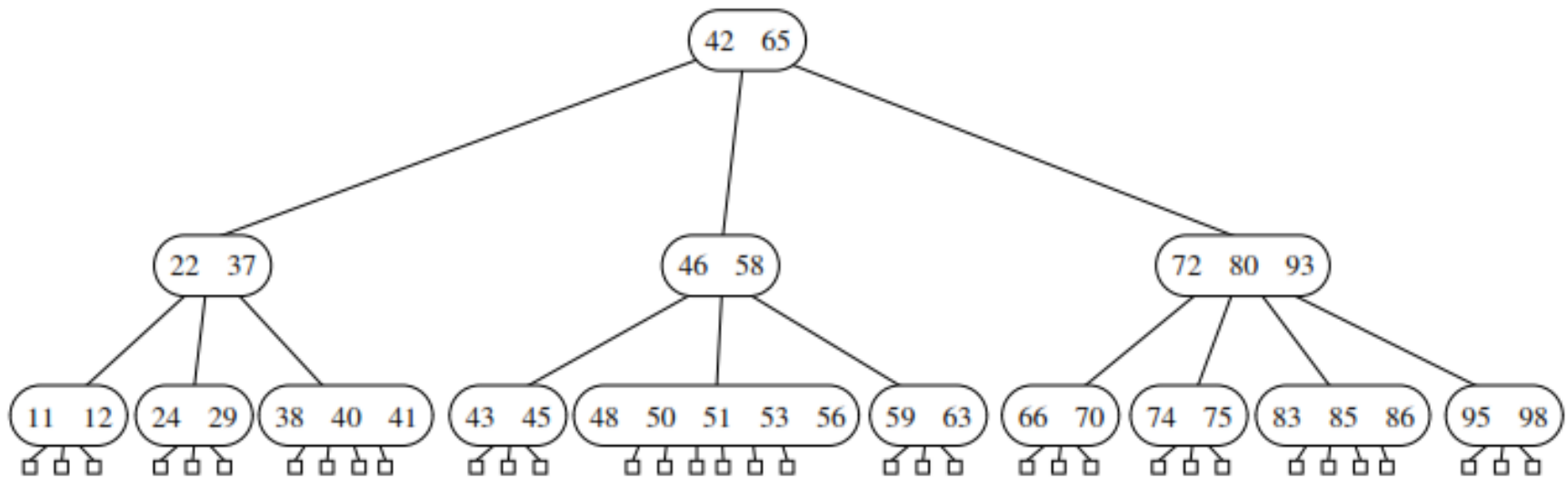
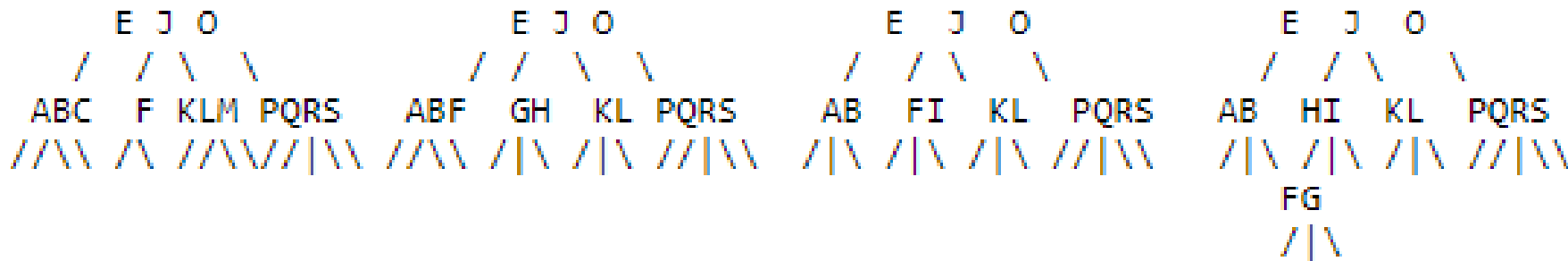


Figure 15.4: A B-tree of order 6.

Exercise

- Which of the following is/are not B-Tree of order 5?



Max number of keys in B-Tree

- What is the MAXIMUM number of KEYS in a B-Tree of order m of height h ?

Complexity of find, insertion and deletion

Performance of (2,4) tree

- The height of a (2,4) tree storing n entries is $O(\log n)$
- A split, transfer, or fusion operation takes $O(1)$ time.
- A search, insertion, or removal of an entry visits $O(\log n)$ nodes.

Space utilization is always 50% or above.

B-Tree vs BST/Binary Search

- $O(\log_d n) \Rightarrow O(\log n / \log d)$

Resources

- Data Structures and Algorithms in Python, by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser. 2013. (1st. ed.). Wiley Publishing
- Open Data Structures (pseudocode edition), by Pat Morin. Available online at <http://opendatastructures.org>
- <https://www.programiz.com/dsa/deletion-from-a-b-tree>
- <https://www.cs.cmu.edu/~ckingsf/bioinfo-lectures/btrees.pdf>
- <https://www.educba.com/b-tree-in-data-structure/>
- [2-3-4 Trees | Algorithm Tutor](#)

Thanks